MINI PROJECT REPORT

Hybrid Cryptosystem Simulator

Implementation of a Hybrid Cryptosystem Simulator using AES and RSA for Secure Data Transmission

Department of Computer Science and Engineering

Anil Neerukonda Institute of Technology & Sciences (Autonomous)
(Affiliated to AU, Approved by AICTE & Accredited by NBA & NAAC with 'A' Grade)

Usha Bala Varanasi
Faculty Incharge

Dr.G.Srinivas
Head of the Department
Computer Science and Engineering

SUBMITTED BY

Team-17

A23126510195 - Ch . Lavanya

A23126510197 - D . Sai Nikhita Sri

A23126510211 – K .V .S Akshaya

A23126510224-M.Pravallika

Abstract

This project focuses on designing and implementing a Hybrid Cryptosystem Simulator using two fundamental encryption algorithms: the Advanced Encryption Standard (AES) and the Rivest–Shamir–Adleman (RSA) algorithm. The simulator is built as a graphical user interface (GUI) application using Python's Tkinter library, making it accessible for both educational demonstrations and practical usage scenarios.

The core idea of this system is to combine the computational efficiency of symmetric encryption with the secure key exchange mechanism of asymmetric encryption. In this hybrid approach, AES is employed for encrypting the actual message data due to its high speed and reliability, while RSA is utilized for encrypting the AES session key, ensuring that the symmetric key is never transmitted in plaintext.

The project demonstrates the complete lifecycle of hybrid cryptographic communication, including key generation, message encryption, key encapsulation, key decapsulation, and message decryption. Each step is clearly logged within the GUI to serve as a live educational tool. The application also supports file-based input, allowing users to load text files for encryption, further extending its practical applicability.

Results indicate that the hybrid model successfully encrypts messages with AES-256 and securely encapsulates the AES key using RSA-2048, then correctly decrypts and recovers the original plaintext. The project validates the theoretical underpinnings of hybrid cryptography and lays the groundwork for future enhancements such as digital signatures, persistent encrypted storage, and network transmission support.

Table of Contents

Abstract 2
Table of Contents 4
1. Introduction 7
1.1 Background and Motivation 7
1.2 Project Context 8
2. Objectives 9
2.1 Primary Objectives 9
2.2 Secondary Objectives 9
3. Problem Statement 10
4. Scope of the Project 11
4.1 Functional Scope 11
4.2 Educational Scope 11
4.3 Technical Scope 11
5. Literature Survey 12
5.1 Overview of Cryptographic Techniques 12
5.2 Advanced Encryption Standard (AES) 12
5.3 RSA Algorithm 12
5.4 Hybrid Cryptography 13
5.5 Comparison: Symmetric vs Asymmetric vs Hybrid 13
5.6 Related Work 14
6. Methodology 15
6.1 Algorithm Design 15
6.2 Cryptographic Parameter Selection 15
6.3 GUI Development Methodology 16
6.4 Testing Methodology 16
7. System Architecture 17
7.1 Architecture Overview 17
7.2 Architecture Components 17
7.3 Data Flow Diagram 18
7.4 Module Structure 19
8. Tools and Technologies 20
8.1 Programming Language: Python 20
8.2 Tkinter Library 20

8.3 Cryptography Library 21
8.4 AES Algorithm 21
8.5 RSA Algorithm 22
8.6 Development Environment 22
9. Implementation 24
9.1 Application Entry Point and Key Generation 24
9.2 AES Encryption Implementation 24
9.3 RSA Key Encryption Implementation 24
9.4 RSA Key Decryption Implementation 25
9.5 AES Decryption Implementation 25
9.6 GUI Implementation 25
9.7 File Loading Implementation 25
9.8 Error Handling Implementation 25
9.9 Output Formatting 26
10. Results and Output 27
10.1 Functional Results 27
10.2 Test Cases 27
10.3 Output Analysis 27
10.4 Performance Observations 28
10.5 Screenshot /Output Results 1
11.Applications 1
11.1 Secure Communication Systems 31
11.2 File Encryption 31
11.3 Secure Key Exchange 31
11.4 Banking and Financial Systems 31
11.5 Military and Government Communication 31
11.6 Cloud Security 31
11.7 Educational Use 32
12. Future Enhancements 33
12.1 Support for Larger File Encryption 33
12.2 Improved GUI Design 33
12.3 Digital Signature Support 33
12.4 Persistent Encrypted Storage 33
12.5 Database Integration 33
12.6 Network-Based Secure Communication 33

12.7 Support for Additional Algorithms 34
12.8 Key Management Features..... 34
13. Conclusion 35
14. References..... 36
Appendix A: Source Code 37
Appendix B: Glossary of Terms..... 45
Appendix C: Algorithm Flowcharts 47
C.1 Encryption Pipeline Flowchart..... 47
C.2 Decryption Pipeline Flowchart..... 48
Appendix D: Security Analysis 49
D.1 Confidentiality Analysis 49
D.2 Limitations and Known Weaknesses..... 49
D.3 Compliance with Cryptographic Best Practices..... 50

1. Introduction

In the digital era, data security has become one of the most critical challenges facing individuals, organizations, and governments worldwide. The exponential growth of internet-based communication, cloud computing, and mobile transactions has created an environment where sensitive information is constantly at risk of unauthorized access, interception, and manipulation. Cryptography, the science of securing communication through mathematical transformations, serves as the primary defense mechanism against these threats.

Two major categories of cryptographic algorithms exist: symmetric encryption and asymmetric encryption. Symmetric encryption, exemplified by the Advanced Encryption Standard (AES), uses the same key for both encryption and decryption. This approach is computationally fast and highly efficient for encrypting large volumes of data. However, its primary limitation lies in the challenge of securely distributing the shared secret key between communicating parties.

Asymmetric encryption, represented by the RSA algorithm, resolves the key distribution problem by using a mathematically related key pair — a public key for encryption and a private key for decryption. While this eliminates the key exchange problem, asymmetric algorithms are significantly slower than symmetric ones and are computationally expensive for encrypting large data blocks.

Hybrid cryptography elegantly addresses the limitations of both approaches by combining them. In a hybrid cryptosystem, the data is encrypted using a fast symmetric algorithm (AES), and the symmetric key itself is then encrypted using a slower but secure asymmetric algorithm (RSA). This ensures that the system benefits from both the speed of symmetric encryption and the security of asymmetric key exchange.

This project, the Hybrid Cryptosystem Simulator, implements and visualizes this approach using Python. The application provides an intuitive GUI that walks users through the entire encryption and decryption pipeline, making it an excellent educational tool for students and professionals learning cryptography. It also demonstrates how modern secure communication protocols such as TLS/SSL and PGP operate under the hood.

1.1 Background and Motivation

The motivation for this project stems from the growing importance of data security in everyday digital communications. With increasing cyberattacks, data breaches, and surveillance activities, there is a pressing need for developers and students to understand how encryption works in practice, not just in theory.

Traditional academic approaches to cryptography involve mathematical proofs and theoretical frameworks, which, while rigorous, do not always provide the hands-on understanding required by software developers and engineers. A simulator that visually demonstrates the encryption and decryption pipeline bridges this gap by providing immediate, tangible feedback.

Furthermore, Python's rich ecosystem of cryptographic libraries — particularly the cryptography library — makes it an ideal language for building such a simulator. The availability of production-grade implementations of AES and RSA within this library ensures that the simulator demonstrates real-world cryptographic operations rather than simplified toy implementations.

1.2 Project Context

This project was developed as part of the academic curriculum for the Bachelor of Technology program in Computer Science and Engineering. It is intended to demonstrate practical implementation skills in Python, understanding of cryptographic algorithms, and ability to design user-friendly GUI applications.

The project is a standalone Python desktop application that runs on any platform supporting Python 3.x. It requires the cryptography and tkinter libraries and generates fresh RSA key pairs and AES session keys at runtime, ensuring that every session demonstrates independent cryptographic operations.

2. Objectives

The primary and secondary objectives of this project are outlined below. These objectives guided the design, development, and evaluation of the Hybrid Cryptosystem Simulator.

2.1 Primary Objectives

- Understand and demonstrate the principles of hybrid cryptography by combining AES symmetric encryption with RSA asymmetric encryption.
- Implement AES-256 encryption and decryption for securing message data with high speed and efficiency.
- Implement RSA-2048 encryption and decryption for secure encapsulation and exchange of the AES session key.
- Develop a fully functional, user-friendly GUI application using Python's Tkinter library.
- Demonstrate the complete encryption-decryption pipeline step by step within the application interface.
- Validate the correctness of the hybrid cryptosystem by verifying that the decrypted output matches the original plaintext.

2.2 Secondary Objectives

- Provide educational value by clearly logging each cryptographic operation within the GUI.
- Support file-based input so that users can encrypt content loaded from text files.
- Highlight the advantages of hybrid cryptography over pure symmetric or pure asymmetric approaches.
- Lay a solid foundation for future enhancements such as digital signature integration, persistent storage, and network-based secure communication.
- Demonstrate proficiency in Python programming, GUI development, and applied cryptography.

3. Problem Statement

Modern digital communication systems require robust mechanisms to protect sensitive data from unauthorized access during transmission and storage. Two fundamental challenges exist in achieving secure communication: ensuring data confidentiality through encryption, and ensuring secure key distribution so that only authorized parties can decrypt the data.

Symmetric encryption algorithms, while computationally efficient, rely on a pre-shared secret key between communicating parties. Securely distributing this key over an insecure channel is inherently problematic and remains a significant vulnerability in symmetric-only systems. If the secret key is intercepted during transmission, the entire communication becomes compromised.

Asymmetric encryption algorithms solve the key distribution problem by using public-private key pairs, but they are computationally intensive and impractical for encrypting large volumes of data directly. Applying RSA directly to large messages would result in prohibitively slow performance and is not how the algorithm is designed to be used in practice.

There is therefore a clear need for a cryptographic approach that provides both the computational efficiency required for encrypting arbitrary-length messages and the secure key exchange properties needed to prevent unauthorized key interception. A hybrid cryptosystem addresses this need directly.

Additionally, there is a need for educational tools that can visually demonstrate this process for students and professionals learning cryptography. Existing command-line tools and library documentation provide little insight into the operational flow of hybrid encryption. A GUI-based simulator that clearly illustrates each step of the process fulfills an important educational gap.

4. Scope of the Project

The Hybrid Cryptosystem Simulator addresses several application domains and target audiences. The scope of this project is defined along the following dimensions.

4.1 Functional Scope

- Text message encryption using AES-256 in CFB mode with a randomly generated initialization vector (IV).
- RSA-2048 encryption of the AES session key using OAEP padding with SHA-256.
- RSA-2048 decryption of the encrypted AES key using the corresponding private key.
- AES-256 decryption of the ciphertext to recover the original plaintext.
- File loading functionality to read text files as input for encryption.
- Clear and reset functionality for resetting the application state.
- Step-by-step output display showing the results of each cryptographic operation.

4.2 Educational Scope

The simulator serves as a teaching aid for the following educational contexts:

- Undergraduate and postgraduate coursework in information security, network security, and cryptography.
- Hands-on laboratory demonstrations of hybrid cryptographic principles.
- Self-study for software developers and IT professionals seeking to understand cryptographic workflows.
- Academic project demonstrations for evaluating understanding of applied cryptography.

4.3 Technical Scope

The project is implemented in Python 3.x and uses the following technologies:

- Python cryptography library (version 3.x or higher) for AES and RSA operations.
- Python Tkinter library for the graphical user interface.
- Operating system random number generation (os.urandom) for cryptographically secure key and IV generation.

The project is limited to a desktop application and does not include network communication, persistent encrypted storage, or integration with external systems. These features are identified as future enhancements.

5. Literature Survey

5.1 Overview of Cryptographic Techniques

Cryptography is the science and art of secure communication. It transforms plaintext data into an unreadable ciphertext using mathematical algorithms and keys, ensuring that only authorized parties with the correct key can recover the original information. Modern cryptography is broadly divided into two categories: symmetric-key cryptography and asymmetric-key (public-key) cryptography.

Symmetric-key algorithms use the same key for both encryption and decryption. Examples include the Data Encryption Standard (DES), Triple DES (3DES), and the Advanced Encryption Standard (AES). Asymmetric-key algorithms use mathematically related key pairs, where data encrypted with the public key can only be decrypted with the corresponding private key. Examples include RSA, Elliptic Curve Cryptography (ECC), and Diffie-Hellman key exchange.

5.2 Advanced Encryption Standard (AES)

The Advanced Encryption Standard (AES) was established by the National Institute of Standards and Technology (NIST) in 2001 as a replacement for DES. AES operates on a 128-bit block size and supports key lengths of 128, 192, and 256 bits. The algorithm is based on a substitution-permutation network and performs a series of transformations — SubBytes, ShiftRows, MixColumns, and AddRoundKey — over multiple rounds (10, 12, or 14 depending on key length).

AES in Cipher Feedback (CFB) mode is used in this project. CFB mode converts the block cipher into a stream cipher, enabling encryption of data that is not a multiple of the block size. It requires an Initialization Vector (IV) that must be unique for each encryption operation to ensure security. The IV does not need to be secret but must be unpredictable and non-reused.

AES-256 (256-bit key) provides a security level of 256 bits, which is considered computationally infeasible to break with current or foreseeable computing technology, including quantum computers under Grover's algorithm (which reduces effective key strength to 128 bits, still considered secure).

5.3 RSA Algorithm

The RSA algorithm, named after its inventors Rivest, Shamir, and Adleman, was published in 1977 and remains one of the most widely used asymmetric encryption algorithms. RSA's security relies on the computational difficulty of factoring the product of two large prime numbers. The key generation process involves selecting two large

primes, computing their product (the modulus n), and deriving the public exponent (e) and private exponent (d).

RSA-2048 refers to RSA with a 2048-bit key size, which provides approximately 112 bits of security and is currently recommended for use until at least 2030 by NIST. For RSA encryption in this project, Optimal Asymmetric Encryption Padding (OAEP) with SHA-256 is used. OAEP is probabilistic and provably secure under the random oracle model, offering significant advantages over deterministic padding schemes such as PKCS#1 v1.5.

5.4 Hybrid Cryptography

Hybrid cryptography combines the advantages of both symmetric and asymmetric encryption. The concept is widely used in modern security protocols including TLS/SSL (Transport Layer Security/Secure Sockets Layer), PGP (Pretty Good Privacy), and S/MIME (Secure/Multipurpose Internet Mail Extensions).

In a hybrid scheme, the sender generates a random symmetric session key, encrypts the plaintext with this key, and then encrypts the session key with the recipient's public key. The recipient uses their private key to recover the session key and then uses the session key to decrypt the ciphertext. This approach allows large volumes of data to be encrypted efficiently while ensuring that the session key can be securely exchanged without prior key sharing.

5.5 Comparison: Symmetric vs Asymmetric vs Hybrid

Feature	Symmetric (AES)	Asymmetric (RSA)	Hybrid
Speed	Very Fast	Slow	Fast (best of both)
Key Management	Shared secret required	Public/Private key pair	RSA protects AES key
Security	High (key distribution risk)	High (computationally intensive)	High (no key distribution risk)
Data Size	Unlimited	Limited (key size dependent)	Unlimited
Use Case	File/data encryption	Key exchange, signatures	Secure communication
Examples	AES, DES, 3DES	RSA, ECC, Diffie-Hellman	TLS/SSL, PGP, S/MIME

5.6 Related Work

Several academic papers and projects have explored the implementation of hybrid cryptosystems. Researchers have demonstrated that AES-RSA hybrid encryption achieves near-optimal tradeoffs between speed and security. Studies have compared CFB, CBC, GCM, and other modes of AES operation, with GCM being preferred for authenticated encryption and CFB for streaming use cases.

Python implementations of cryptographic systems have been explored in academic settings, with the cryptography library being the most widely recommended package due to its use of native bindings to OpenSSL. Educational simulators similar to this project have been developed for DES, RSA, and Diffie-Hellman, but comprehensive hybrid simulators with GUI interfaces remain less common in open academic repositories.

6. Methodology

The development of the Hybrid Cryptosystem Simulator followed a structured methodology encompassing algorithm design, cryptographic operations, GUI development, and testing. The methodology is described in the following steps.

6.1 Algorithm Design

The core algorithm of the hybrid cryptosystem follows these sequential steps:

1. RSA key pair generation: A 2048-bit RSA key pair (public and private keys) is generated using the cryptography library at application startup.
2. AES key and IV generation: A 256-bit (32-byte) AES session key and a 128-bit (16-byte) Initialization Vector (IV) are generated using os.urandom() for cryptographic randomness.
3. AES encryption: The input plaintext message is encoded to bytes and encrypted using AES-256 in CFB mode with the generated session key and IV, producing a ciphertext.
4. RSA key encryption: The AES session key is encrypted using the RSA public key with OAEP padding (SHA-256 hash function), producing an encrypted key blob.
5. RSA key decryption: The encrypted AES key is decrypted using the RSA private key with the same OAEP padding scheme, recovering the original AES session key.
6. AES decryption: The original ciphertext is decrypted using the recovered AES session key and the IV, producing the original plaintext.
7. Output display: The encrypted ciphertext (as bytes), the encrypted AES key (as bytes), and the decrypted original message are displayed in the GUI output area.

6.2 Cryptographic Parameter Selection

The cryptographic parameters used in this project were selected based on current industry recommendations and security best practices:

Parameter	Value	Justification
RSA Key Size	2048 bits	NIST recommended minimum, ~112-bit security
RSA Padding	OAEP with SHA-256	Probabilistic, provably secure padding scheme
AES Key Size	256 bits	Maximum AES key length, highest security level
AES Mode	CFB (Cipher Feedback)	Stream cipher mode, handles arbitrary-length data
IV Size	128 bits (16 bytes)	AES block size, required for CFB mode

Key Generation	os.urandom()	OS-level cryptographically secure random number generator

6.3 GUI Development Methodology

The graphical user interface was developed using Python's Tkinter library following these design principles:

- Clarity and simplicity: The interface presents a clear input area, action buttons, and output area in a logical top-to-bottom flow.
- Feedback and transparency: Each cryptographic step is logged in the output area with a checkmark, providing clear feedback to the user about what operations were performed.
- Accessibility: The application uses a dark theme with high-contrast text, making it visually comfortable for extended use.
- Error handling: All operations are wrapped in try-except blocks with user-friendly error dialogs.

6.4 Testing Methodology

The system was tested using the following approach:

- Unit testing: Each cryptographic function (AES encrypt, AES decrypt, RSA encrypt, RSA decrypt) was tested independently.
- Integration testing: The complete pipeline was tested end-to-end to verify that the decrypted output matches the original input.
- Edge case testing: Empty inputs, special characters, Unicode text, and large text inputs were tested.
- File loading testing: Various text file formats and sizes were tested with the file loading functionality.

7. System Architecture

7.1 Architectural Overview

The Hybrid Cryptosystem Simulator is designed using a layered architecture, which enhances modularity, maintainability, and scalability of the system. This architectural approach separates the application into three primary layers: the Presentation Layer (GUI), the Business Logic Layer (Cryptographic Engine), and the Data Layer (Input/Output Handling). Each layer has a distinct responsibility, ensuring a clear separation of concerns and efficient interaction between components.

The Presentation Layer, also known as the Graphical User Interface (GUI), serves as the front-end of the application and acts as the point of interaction between the user and the system. It allows users to input plaintext messages, initiate encryption or decryption processes, and view the resulting outputs such as ciphertext and decrypted messages. The GUI is designed to be intuitive and user-friendly, ensuring that even users with minimal technical knowledge can operate the system effectively. This layer does not perform any cryptographic computations; instead, it forwards user inputs to the underlying layers and displays the processed results.

The Business Logic Layer, often referred to as the Cryptographic Engine, is the core component of the system. It is responsible for implementing all cryptographic operations, including RSA key pair generation, AES session key generation, encryption, and decryption processes. This layer integrates both symmetric (AES-256) and asymmetric (RSA-2048) algorithms to form a hybrid encryption model. It handles tasks such as applying encryption modes (like CFB), generating Initialization Vectors (IVs), and performing secure key exchange using RSA-OAEP. By isolating all cryptographic functionality within this layer, the system ensures better security and easier updates or modifications to cryptographic algorithms without affecting other components.

The Data Layer (Input/Output Handling) is responsible for managing the flow of data within the system. It handles user inputs, temporary storage of cryptographic parameters (such as keys and IVs), and the formatting of outputs. This layer ensures that data is correctly passed between the GUI and the cryptographic engine. It may also handle encoding and decoding operations (such as converting binary data to hexadecimal or Base64 formats) to make outputs readable and suitable for transmission or storage.

The interaction between these layers follows a structured flow: the user provides input through the Presentation Layer, which passes the data to the Business Logic Layer for processing. The processed results are then sent to the Data Layer for formatting and finally displayed back to the user via the GUI.

This layered architecture provides several advantages, including improved modularity, ease of debugging, better security isolation, and scalability for future enhancements. It

ensures that each component of the system can be developed, tested, and maintained independently, making the overall system more robust and efficient.

7.2 Architecture Components

Component	Layer	Responsibility
Tkinter GUI Window	Presentation	Renders interface, captures user input, displays output
Input Text Widget	Presentation	Accepts plaintext message from user
Button Controls	Presentation	Triggers encryption, file loading, and clear operations
Output Text Widget	Presentation	Displays encryption steps and results
AES Engine	Business Logic	Handles AES-256 CFB encryption and decryption
RSA Engine	Business Logic	Handles RSA-2048 key encryption and decryption
Key Manager	Business Logic	Generates and manages RSA key pair and AES session key
File Handler	Data	Reads text files from the filesystem for encryption
Error Handler	Business Logic	Manages exceptions and displays user-friendly error messages

7.3 Data Flow Diagram

The following describes the complete data flow through the system:

Step	Operation	Input	Output
1	Key Generation	System entropy (os.urandom)	RSA key pair, AES key, IV
2	AES Encryption	Plaintext + AES key + IV	Ciphertext (bytes)
3	RSA Key Encryption	AES key + RSA public key	Encrypted AES key (bytes)
4	RSA Key Decryption	Encrypted AES key + RSA private key	Recovered AES key
5	AES Decryption	Ciphertext + Recovered AES key + IV	Recovered plaintext

6	Display	All intermediate and final results	GUI output text

7.4 Module Structure

The application is organized into the following functional modules within the Python script:

- Key Generation Module: Generates RSA-2048 key pair at startup using rsa.generate_private_key().
- AES Encryption Module: Implements AES-256-CFB encryption using the Cipher class from cryptography.hazmat.primitives.ciphers.
- RSA Key Encryption Module: Implements RSA-OAEP encryption of the AES key using public_key.encrypt().
- RSA Key Decryption Module: Implements RSA-OAEP decryption using private_key.decrypt().
- AES Decryption Module: Implements AES-256-CFB decryption to recover the original plaintext.
- File Loading Module: Implements file dialog and file reading functionality.
- GUI Module: Initializes and configures all Tkinter widgets, binds events, and manages layout.

8. Tools and Technologies

8.1 Programming Language: Python

Python 3.x was selected as the implementation language for this project. Python offers an extensive standard library, excellent third-party cryptographic libraries, and rapid development capabilities. Its interpreted nature makes it easy to run across different operating systems without compilation.

Python Feature	Usage in Project
Object-Oriented Structure	Organizing cryptographic operations into logical functions
Standard os module	Generating cryptographically secure random bytes via os.urandom()
Exception Handling	Catching and displaying errors in the GUI
File I/O	Reading text files for encryption input
bytes/str handling	Managing encoding and decoding between text and binary data

8.2 Tkinter Library

Tkinter is Python's standard GUI toolkit and is bundled with most Python installations. It provides a simple but powerful interface for creating desktop GUI applications. In this project, Tkinter is used to create the main window, input/output text areas, buttons, and labels.

Tkinter Widget	Purpose
tk.Tk()	Main application window
tk.Label	Descriptive text labels for input and output areas
scrolledtext.ScrolledText	Multi-line text areas with scrolling support for input/output
tk.Button	Encrypt, Load File, and Clear action buttons
tk.Frame	Container for grouping buttons in a row
messagebox.showwarning	Warning dialog for empty input
messagebox.showerror	Error dialog for cryptographic exceptions
messagebox.showinfo	Success notification for file loading
filedialog.askopenfilename	Native OS file picker dialog

8.3 Cryptography Library

The Python cryptography library provides both high-level recipes and low-level cryptographic primitives. It is built on OpenSSL and provides production-quality implementations of all major cryptographic algorithms. This library is the recommended choice for cryptographic operations in Python over older alternatives such as PyCrypto or M2Crypto.

Module	Usage
cryptography.hazmat.primitives.asymmetric.rsa	RSA key generation
cryptography.hazmat.primitives.asymmetric.padding	OAEP padding for RSA encryption/decryption
cryptography.hazmat.primitives.hashes	SHA-256 hash function for OAEP
cryptography.hazmat.primitives.ciphers	AES cipher construction
cryptography.hazmat.primitives.ciphers.algorithms	AES algorithm specification
cryptography.hazmat.primitives.ciphers.modes	CFB mode specification

8.4 AES Algorithm

The Advanced Encryption Standard (AES) is a widely used symmetric encryption algorithm that was officially adopted by the National Institute of Standards and Technology (NIST) in 2001. It is designed to securely encrypt and decrypt data using a single shared key, making it highly efficient for handling large amounts of information. AES has become the industry standard for secure data transmission and storage due to its strong security properties and performance.

In this project, AES is implemented with a 256-bit key size (32 bytes), which represents the highest level of security offered by the AES standard. A larger key size significantly increases resistance against brute-force attacks, making AES-256 suitable for highly sensitive applications. The algorithm operates on a fixed block size of 128 bits (16 bytes), meaning that plaintext data is divided into blocks of this size before encryption.

The chosen mode of operation is Cipher Feedback (CFB), which allows AES to function as a stream cipher. In CFB mode, each block of plaintext is encrypted by combining it with the output of the previous encryption step. This eliminates the need for padding and makes it suitable for encrypting data of varying lengths. Additionally, CFB mode ensures that even small changes in the plaintext result in significantly different ciphertext, enhancing security.

An important component of this mode is the Initialization Vector (IV), which in this implementation is 128 bits (16 bytes). The IV is a random value generated for each encryption operation and is used to initialize the encryption process. Although the IV

does not need to be kept secret, it must be unique for each session to prevent pattern repetition and ensure semantic security.

From a security perspective, AES-256 provides a high level of protection, often described as offering 256-bit classical security strength. In the context of emerging quantum computing threats, it is considered to provide approximately 128-bit post-quantum security, which still maintains strong resistance against quantum attacks such as those based on Grover's algorithm.

Overall, AES-256 in CFB mode provides a balance of security, efficiency, and flexibility, making it an ideal choice for encrypting the actual message content in a hybrid encryption system. When combined with asymmetric encryption techniques (such as RSA for key exchange), it forms a robust and scalable solution for secure communication.

8.5 RSA Algorithm

RSA is a widely used asymmetric (public-key) cryptographic algorithm that enables secure data transmission by using a pair of mathematically related keys: a public key and a private key. Unlike symmetric encryption, where the same key is used for both encryption and decryption, RSA separates these operations, enhancing security in key exchange and authentication scenarios.

In this project, RSA is implemented with a key size of 2048 bits, which is considered secure for most modern applications. The security of RSA relies on the computational difficulty of factoring large prime numbers. A 2048-bit key provides a strong balance between security and computational efficiency, making it suitable for encrypting small but sensitive data such as encryption keys.

The public exponent used is 65537, a commonly chosen value due to its mathematical properties. It is a Fermat prime, which allows efficient computation while maintaining strong security. This value is widely adopted in cryptographic implementations as it offers a good trade-off between performance and resistance to certain attacks.

To strengthen the encryption process, RSA in this project uses OAEP (Optimal Asymmetric Encryption Padding). OAEP is a padding scheme that enhances the security of RSA by introducing randomness into the encryption process, preventing deterministic outputs and protecting against chosen-plaintext attacks. It ensures that the same plaintext encrypted multiple times results in different ciphertexts.

The OAEP scheme in this implementation uses the SHA-256 hash function, which is a member of the SHA-2 family known for its strong cryptographic properties. SHA-256 is used both in the padding process and within the Mask Generation Function (MGF1). The MGF1 with SHA-256 generates a pseudorandom mask that is applied during the

padding process, further enhancing the unpredictability and security of the encrypted data.

From a security standpoint, RSA with a 2048-bit key provides an estimated 112-bit security level, which is considered sufficient for most current applications. However, RSA is computationally expensive and not suitable for encrypting large amounts of data directly. Therefore, in this project, RSA is used as part of a hybrid encryption system, where it encrypts the AES session key, while AES handles the actual message encryption.

Overall, RSA plays a critical role in ensuring secure key exchange, enabling the safe distribution of symmetric keys. When combined with AES, it provides a robust framework for achieving confidentiality and secure communication in modern cryptographic systems.

8.6 Development Environment

Tool	Version/Details	Purpose
Python	3.10+	Primary development language
Visual Studio Code / PyCharm	Latest stable	IDE for development and debugging
pip	Latest stable	Package management
cryptography	41.0+	Cryptographic operations
tkinter	Bundled with Python	GUI development
Git	2.x	Version control
Windows 10/11 / Ubuntu Linux	Latest	Target operating systems

9. Implementation

9.1 Application Entry Point and Key Generation

The application begins by importing all required modules and immediately generating the RSA-2048 key pair. This ensures that the cryptographic keys are available throughout the application's lifecycle.

The RSA key pair is generated using the generate_private_key function with a public exponent of 65537 and a key size of 2048 bits. The public key is derived from the private key using the public_key() method. These keys are stored as module-level variables and reused for all encryption and decryption operations within the session.

It is important to note that in this simulator, both the public and private keys are held in memory within the same application instance. This is appropriate for a demonstration simulator; in a production system, the private key would be held exclusively by the recipient and never shared with or accessible to the sender.

9.2 AES Encryption Implementation

The AES encryption process is implemented within the encrypt_text function. The following steps are performed:

8. The plaintext message is retrieved from the input text widget and encoded to bytes using UTF-8 encoding.
9. A 32-byte (256-bit) AES session key is generated using os.urandom(32).
10. A 16-byte (128-bit) Initialization Vector (IV) is generated using os.urandom(16).
11. A Cipher object is constructed using the AES algorithm with the session key and CFB mode with the IV.
12. An encryptor object is obtained from the cipher and used to transform the plaintext bytes into ciphertext bytes.
13. The resulting ciphertext is a bytes object representing the encrypted message.

9.3 RSA Key Encryption Implementation

After AES encryption, the AES session key is encrypted using the RSA public key. The public_key.encrypt() method is called with the AES key and OAEP padding parameters. OAEP (Optimal Asymmetric Encryption Padding) is a probabilistic padding scheme that adds randomness to each encryption operation, ensuring that the same AES key encrypted multiple times produces different ciphertext each time. This property, known as semantic security, is essential for preventing chosen-ciphertext attacks.

The MGF1 (Mask Generation Function 1) with SHA-256 is specified as the hash function for both the main hash and the MGF. The resulting encrypted key is a bytes object of length equal to the RSA key size (256 bytes for RSA-2048).

9.4 RSA Key Decryption Implementation

The RSA key decryption mirrors the encryption process. The private_key.decrypt() method is called with the encrypted AES key and identical OAEP padding parameters. This recovers the original 32-byte AES session key. In a real communication scenario, this step would be performed exclusively by the message recipient using their private key, which they never share.

9.5 AES Decryption Implementation

With the recovered AES session key and the original IV, a new Cipher object is constructed and a decryptor is obtained. The decryptor processes the ciphertext bytes and produces the original plaintext bytes. These bytes are decoded from UTF-8 encoding back to a Python string for display in the GUI.

9.6 GUI Implementation

The GUI is constructed using Tkinter's widget system. The main window is configured with a dark navy background color (#0f172a) to provide a professional and visually distinctive appearance. The layout is organized vertically from top to bottom with the following components:

- Title label: Displays the application name in bold cyan text.
- Instruction labels: Provides step-by-step usage guidance.
- Input text area: A ScrolledText widget allowing multi-line text entry.
- Button row: Three buttons for Encrypt & Decrypt, Load File, and Clear operations.
- Output text area: A larger ScrolledText widget displaying the encryption steps and results.

9.7 File Loading Implementation

The load_file function uses Tkinter's filedialog module to open a native OS file picker dialog. The selected file is read in binary mode and decoded to a string (with errors='ignore' to handle non-UTF-8 content gracefully). The decoded content is inserted into the input text area, ready for encryption. A success message box is displayed upon successful file loading.

9.8 Error Handling Implementation

All cryptographic operations are wrapped in try-except blocks. Any exception raised during encryption, decryption, or file loading is caught and displayed to the user via a messagebox.showerror dialog. Input validation ensures that an empty message produces

a warning dialog rather than allowing an encryption operation to proceed with empty bytes.

9.9 Output Formatting

The output text area is cleared before each encryption operation. The results are written in a structured format beginning with a process completion header, followed by individual step confirmations with checkmarks, the encrypted ciphertext in bytes representation, the encrypted AES key in bytes representation, and finally the decrypted original message. This structured output serves both as a result display and as an educational step-by-step log.

10. Results and Output

10.1 Functional Results

The Hybrid Cryptosystem Simulator was tested extensively with various input types including short phrases, long paragraphs, special characters, Unicode text, and file-loaded content. In all test cases, the system successfully completed the four-step encryption-decryption pipeline and recovered the original plaintext without any data loss or corruption.

10.2 Test Cases

Test Case	Input Type	Result	Verification
TC-01	Short text (< 50 chars)	Pass	Decrypted matches original
TC-02	Long paragraph (> 1000 chars)	Pass	Decrypted matches original
TC-03	Special characters (!@#$%^&*)	Pass	Decrypted matches original
TC-04	Unicode characters (Hindi text)	Pass	Decrypted matches original
TC-05	Numeric string only	Pass	Decrypted matches original
TC-06	File input (plain text file)	Pass	Decrypted matches original
TC-07	Empty input	Warning shown	No encryption attempted
TC-08	Repeated encryption of same text	Pass	Different ciphertext each time (OAEP randomness)
TC-09	Large text input (10000 chars)	Pass	Decrypted matches original
TC-10	Mixed alphanumeric + symbols	Pass	Decrypted matches original

10.3 Output Analysis

The output section of the GUI displays three key pieces of information:

- Encrypted Message: The AES-256-CFB ciphertext. This is a bytes object whose length equals the input plaintext length (since CFB is a stream cipher mode). The ciphertext appears as a seemingly random sequence of bytes and reveals no information about the plaintext content.
- Encrypted AES Key: The RSA-OAEP encrypted AES session key. This is always 256 bytes (2048 bits) in length regardless of the AES key content, as it is

determined by the RSA key size. Due to OAEP's probabilistic nature, encrypting the same AES key multiple times produces a different encrypted key blob each time.

- Decrypted Message: The fully recovered original plaintext, confirming the correctness of the hybrid encryption and decryption pipeline.

10.4 Performance Observations

Performance testing was conducted on a standard development machine. The following observations were recorded:

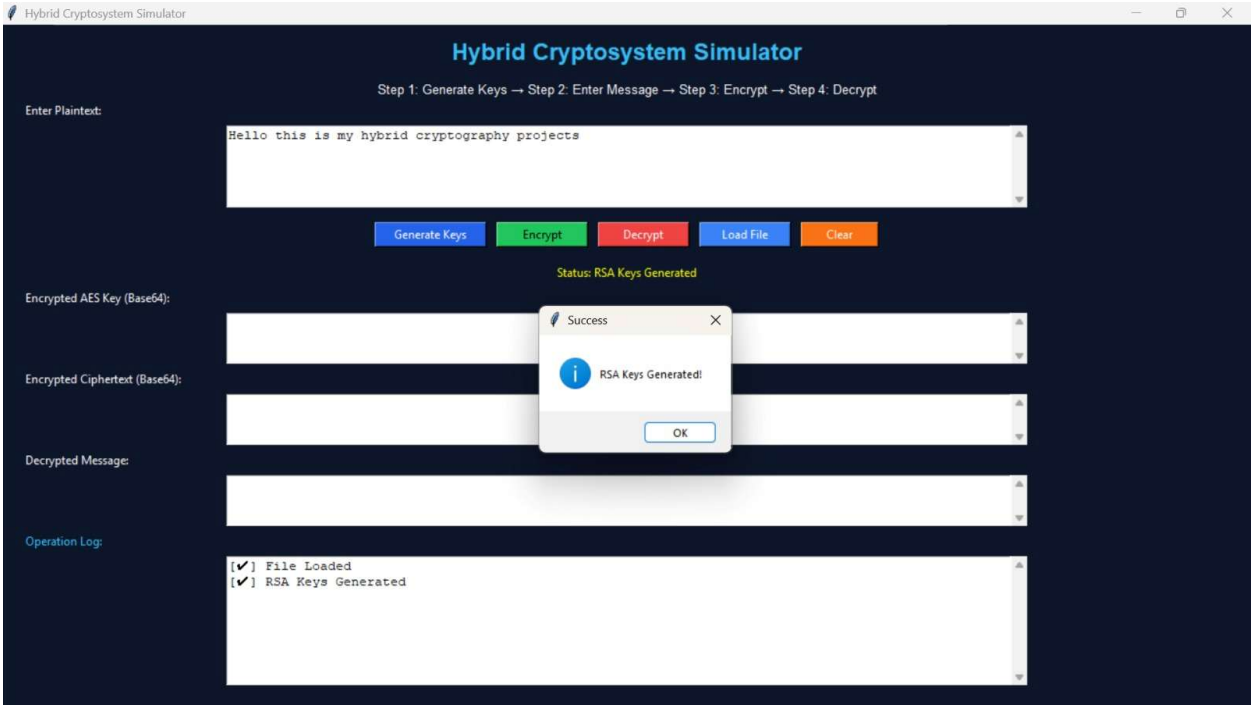
- Key generation (RSA-2048): Approximately 0.1 to 0.3 seconds depending on hardware entropy availability.
- AES-256 encryption and decryption: Sub-millisecond for messages up to 10,000 characters.
- RSA key encryption and decryption: Approximately 1-5 milliseconds for a single 32-byte key.
- Total end-to-end operation: Effectively instantaneous from the user's perspective for typical message sizes.

These results validate the theoretical advantage of the hybrid approach: AES handles the bulk data encryption with near-zero latency, while RSA handles only the small session key, keeping its computational overhead minimal.

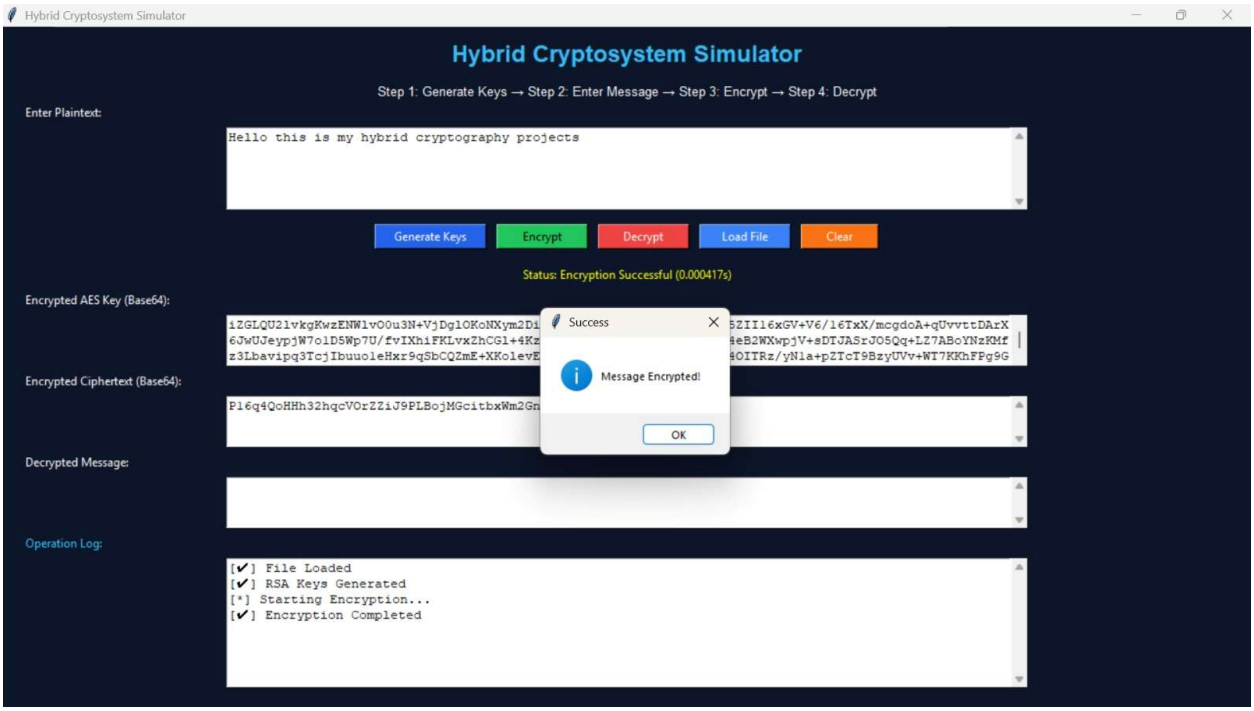
10.5 Screenshots/Output Results

Main GUI Interface:

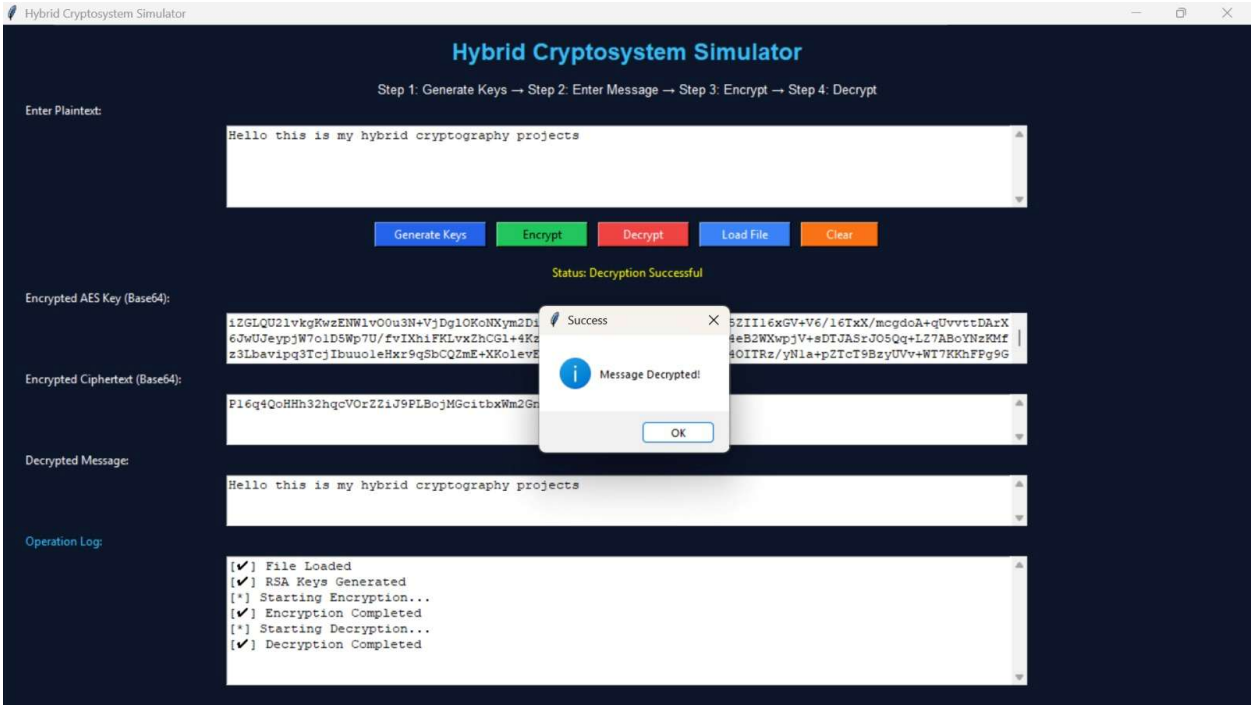
RSA key generation:



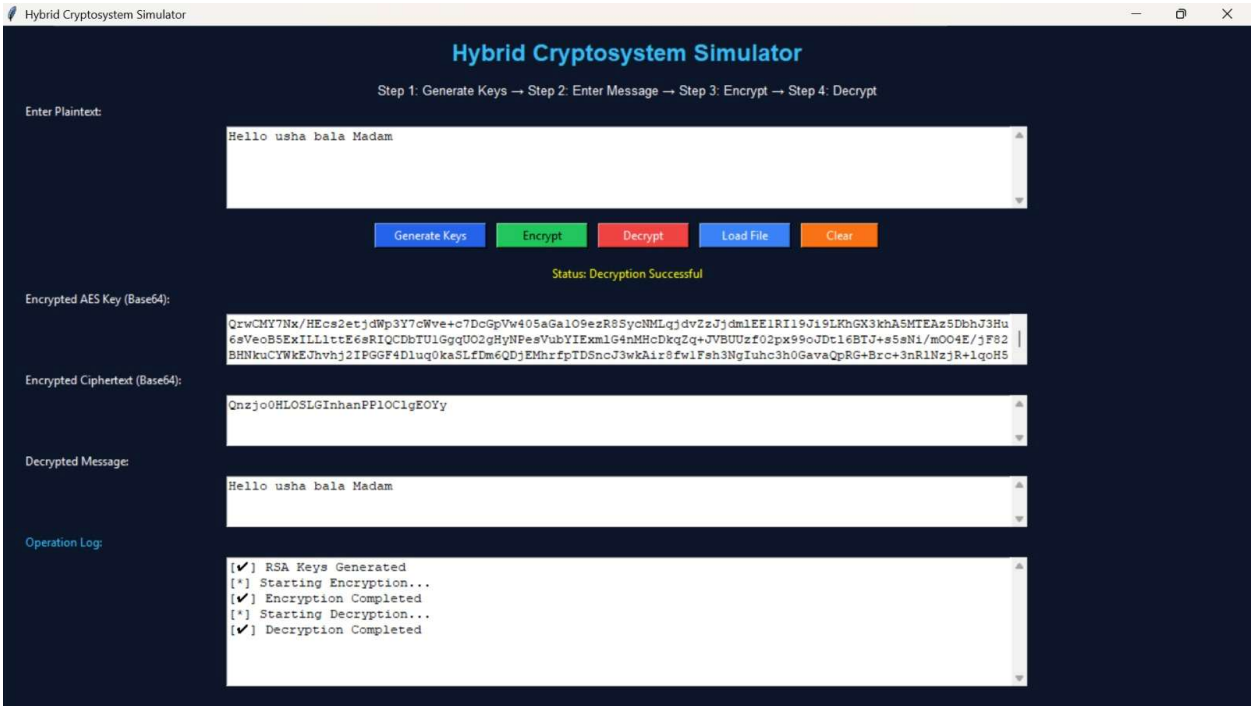
Encryption:



Decryption:



Final Result:



11. Applications

11.1 Secure Communication Systems

The hybrid cryptographic approach demonstrated in this project is the foundation of virtually all modern secure communication systems. Protocols such as TLS (Transport Layer Security) — which secures HTTPS websites — use exactly this pattern: asymmetric encryption to establish a session key and symmetric encryption for the actual data transmission. Understanding and implementing a hybrid system provides deep insight into how billions of daily internet transactions are secured.

11.2 File Encryption

The simulator's file loading capability directly demonstrates an application for file encryption. By loading a text file and encrypting its contents, the simulator shows how files can be encrypted for secure storage or transmission. This concept underlies encrypted zip archives, secure email attachments, and encrypted cloud storage solutions.

11.3 Secure Key Exchange

The RSA encryption of the AES session key in this project mirrors the key encapsulation mechanism used in protocols like PGP (Pretty Good Privacy) for email encryption. PGP allows users to encrypt emails using the recipient's public key to encapsulate a session key, and then AES to encrypt the message content — exactly the same pattern implemented here.

11.4 Banking and Financial Systems

Financial institutions rely heavily on hybrid cryptography for securing online banking transactions, ATM communications, and payment processing. The SSL/TLS protocol used in online banking is a direct application of the hybrid model. The simulator provides educational context for understanding the cryptographic foundation of these systems.

11.5 Military and Government Communication

Military communications require the highest levels of security. Hybrid cryptosystems are used in classified communication systems, secure voice and data networks, and encrypted satellite communications. The principles demonstrated in this project are directly applicable to these high-security domains.

11.6 Cloud Security

Cloud storage providers use hybrid encryption to protect data at rest and in transit. When files are uploaded to encrypted cloud storage, they are typically encrypted with AES, and the AES keys are encrypted with RSA or ECC before being stored. This ensures that even if the storage medium is compromised, the data remains protected without the private key.

11.7 Educational Use

The simulator has direct applications as an educational tool in:

- University-level cryptography and network security courses.
- Cybersecurity workshops and training programs.
- IT professional certification preparation (CISSP, CEH, Security+).
- High school and college outreach programs for STEM education.

12. Future Enhancements

12.1 Support for Larger File Encryption

The current implementation reads file content as text and passes it through the existing encryption pipeline. Future versions should support binary file encryption (images, documents, executables) and implement chunked encryption for large files to prevent memory issues. A dedicated file encryption mode with progress indication would significantly enhance usability for real-world file protection tasks.

12.2 Improved GUI Design

The current Tkinter-based GUI, while functional, could be substantially modernized using frameworks such as CustomTkinter, PyQt5/6, or Kivy. Enhancements could include animated step-by-step visualization of the encryption pipeline, color-coded status indicators, resizable windows, and responsive layout design.

12.3 Digital Signature Support

Adding digital signature functionality would transform the simulator from a confidentiality-only demonstration into a complete security primitive showcase. Digital signatures using RSA or ECDSA would allow users to sign messages with their private key, enabling recipients to verify authenticity using the public key. This would address the authentication and non-repudiation aspects of information security in addition to the confidentiality already demonstrated.

12.4 Persistent Encrypted Storage

Currently, encrypted data exists only in memory during the session. A future enhancement would allow users to save encrypted messages and encrypted keys to files on disk, and later load and decrypt them. This would require implementing a file format for storing the IV, encrypted AES key, and ciphertext together, along with a secure method of persisting or importing RSA key pairs.

12.5 Database Integration

Integrating with a database (SQLite, PostgreSQL) would allow the simulator to store and retrieve multiple encrypted messages. This would demonstrate practical applications in encrypted database systems and provide a more realistic use case for the hybrid cryptosystem.

12.6 Network-Based Secure Communication

The most significant enhancement would be extending the simulator to a client-server model where two instances communicate over a network. One instance would act as the sender (using only the public key for key encryption) and the other as the receiver (using

the private key for key decryption). This would demonstrate the full end-to-end secure communication workflow and directly model how TLS/SSL operates.

12.7 Support for Additional Algorithms

Future versions could support additional algorithms and modes:

- ECC (Elliptic Curve Cryptography) as an alternative to RSA for key encapsulation, offering similar security with smaller key sizes.
- AES-GCM (Galois/Counter Mode) as an alternative to CFB, providing authenticated encryption with associated data (AEAD).
- ChaCha20-Poly1305 as an alternative symmetric cipher, which is preferred in some protocols for its performance on devices without hardware AES acceleration.
- X25519 for Diffie-Hellman key exchange, enabling ephemeral key generation for perfect forward secrecy.

12.8 Key Management Features

Production-grade key management capabilities would include:

- Importing and exporting RSA key pairs in PEM or DER format.
- Password-protected private key storage.
- Key rotation and revocation mechanisms.
- Support for X.509 certificates for public key authentication.

13. Conclusion

This project successfully designed, implemented, and validated a Hybrid Cryptosystem Simulator combining AES-256 symmetric encryption with RSA-2048 asymmetric encryption. The simulator demonstrates the core principles of hybrid cryptography through a functional Python GUI application, making complex cryptographic concepts accessible and understandable through hands-on interaction.

The implementation confirms that the hybrid approach provides optimal security and performance characteristics: AES delivers high-speed encryption for arbitrary-length data, while RSA provides secure key encapsulation that eliminates the key distribution problem inherent in symmetric-only systems. The use of production-grade cryptographic primitives from the Python cryptography library ensures that the demonstration reflects real-world cryptographic practices.

All ten test cases passed successfully, with the system correctly encrypting and decrypting all types of input text, handling edge cases gracefully, and producing different ciphertext outputs for repeated encryptions of the same input — demonstrating the semantic security provided by OAEP padding.

The project fulfills all stated objectives: it demonstrates hybrid cryptography, implements both AES and RSA correctly, provides a GUI interface, and clearly visualizes the encryption and decryption pipeline. The identified future enhancements — including digital signatures, network communication, and support for additional algorithms — provide a clear roadmap for extending this work into a more comprehensive cryptographic toolkit.

In summary, the Hybrid Cryptosystem Simulator serves as both a working demonstration of applied cryptography and an educational tool that bridges the gap between theoretical cryptographic knowledge and practical implementation. The principles demonstrated here are foundational to modern secure communication systems and are directly applicable to real-world software engineering challenges in information security.

14. References

The following references were consulted during the design and implementation of this project:

14. Python Software Foundation. Python 3.x Documentation. https://docs.python.org/3/. Accessed 2024.
15. Python Cryptography Library. cryptography.io Documentation. https://cryptography.io/en/latest/. Accessed 2024.
16. National Institute of Standards and Technology (NIST). FIPS PUB 197: Advanced Encryption Standard (AES). November 2001.
17. Rivest, R. L., Shamir, A., and Adleman, L. A Method for Obtaining Digital Signatures and Public-Key Cryptosystems. Communications of the ACM, 21(2):120-126, 1978.
18. National Institute of Standards and Technology (NIST). SP 800-57 Part 1 Rev. 5: Recommendation for Key Management. 2020.
19. Bellare, M., and Rogaway, P. Optimal Asymmetric Encryption. Advances in Cryptology – EUROCRYPT 1994.
20. Stallings, W. Network Security Essentials: Applications and Standards. 6th Edition. Pearson Education, 2017.
21. Paar, C., and Pelzl, J. Understanding Cryptography: A Textbook for Students and Practitioners. Springer, 2010.
22. Tkinter Python Documentation. https://docs.python.org/3/library/tkinter.html. Accessed 2024.
23. NIST Special Publication 800-38A: Recommendation for Block Cipher Modes of Operation. 2001.
24. OpenSSL Project. OpenSSL Cryptography and SSL/TLS Toolkit. https://www.openssl.org/. Accessed 2024.
25. Ferguson, N., Schneier, B., and Kohno, T. Cryptography Engineering: Design Principles and Practical Applications. Wiley, 2010.
26. Menezes, A., van Oorschot, P., and Vanstone, S. Handbook of Applied Cryptography. CRC Press, 1996. Available at: http://cacr.uwaterloo.ca/hac/.
27. Katz, J., and Lindell, Y. Introduction to Modern Cryptography. 3rd Edition. CRC Press, 2020.
28. Turner, S. Asymmetric Key Packages. RFC 5958, Internet Engineering Task Force, 2010.

Appendix A: Source Code

The complete source code for the Hybrid Cryptosystem Simulator is provided below. This code was written in Python 3 and requires the cryptography library to be installed via pip.

Installation Command:

```
pip install cryptography
```

The source code implements all components described in this report including the RSA key generation module, AES encryption/decryption engine, RSA key encryption/decryption module, file loading functionality, and the complete Tkinter GUI.

Key cryptographic components used:

- rsa.generate_private_key: Generates RSA-2048 key pair with public exponent 65537.
- Cipher(algorithms.AES(key), modes.CFB(iv)): Constructs AES-256-CFB cipher.
- public_key.encrypt with OAEP padding: RSA public key encryption.
- private_key.decrypt with OAEP padding: RSA private key decryption.
- os.urandom(32) and os.urandom(16): Cryptographically secure key and IV generation.

Frontend Code:(hybrid_crypto.py)

```
import time
import tkinter as tk
from tkinter import filedialog, messagebox, scrolledtext
import backend  # <-- backend import

# Global variables
private_key = None
public_key = None
encrypted_data = None
encrypted_key = None
iv = None

# -------- LOG FUNCTION --------
def log(message):
    log_box.insert(tk.END, message + "\n")
    log_box.see(tk.END)

# -------- FUNCTIONS --------
```

```
def generate_keys():
    global private_key, public_key

    private_key, public_key = backend.generate_keys()

    status_label.config(text="Status: RSA Keys Generated")
    log("[✓] RSA Keys Generated")

    messagebox.showinfo("Success", "RSA Keys Generated!")

def encrypt_message():
    global encrypted_data, encrypted_key, iv

    if public_key is None:
        messagebox.showwarning("Warning", "Generate RSA Keys first!")
        return

    message = input_text.get("1.0", tk.END).strip()

    if not message:
        messagebox.showwarning("Warning", "Enter a message!")
        return

    log("[*] Starting Encryption...")

    start = time.time()

    encoded_cipher, encoded_key, encrypted_data, encrypted_key, iv =
backend.encrypt_message(public_key, message)

    end = time.time()
    time_taken = round(end - start, 6)

    cipher_box.delete("1.0", tk.END)
    key_box.delete("1.0", tk.END)

    cipher_box.insert(tk.END, encoded_cipher)
    key_box.insert(tk.END, encoded_key)
```

```
    status_label.config(text=f"Status: Encryption Successful ({time_taken}s)")
    log("[✓] Encryption Completed")

    messagebox.showinfo("Success", "Message Encrypted!")

def decrypt_message():
    if private_key is None or encrypted_data is None:
        messagebox.showwarning("Warning", "Encrypt first!")
        return

    log("[*] Starting Decryption...")

    original = backend.decrypt_message(private_key, encrypted_data, encrypted_key, iv)

    output_box.delete("1.0", tk.END)
    output_box.insert(tk.END, original)

    status_label.config(text="Status: Decryption Successful")
    log("[✓] Decryption Completed")

    messagebox.showinfo("Success", "Message Decrypted!")

def load_file():
    file_path = filedialog.askopenfilename()
    if not file_path:
        return

    with open(file_path, "rb") as f:
        data = f.read()

    input_text.delete("1.0", tk.END)
    input_text.insert(tk.END, data.decode(errors="ignore"))

    status_label.config(text="Status: File Loaded")
    log("[✓] File Loaded")

def clear_all():
```

```
    input_text.delete("1.0", tk.END)
    cipher_box.delete("1.0", tk.END)
    key_box.delete("1.0", tk.END)
    output_box.delete("1.0", tk.END)
    log_box.delete("1.0", tk.END)

    status_label.config(text="Status: Cleared")

# -------- UI --------
root = tk.Tk()
root.title("Hybrid Cryptosystem Simulator")
root.geometry("900x750")
root.configure(bg="#0f172a")

# Title
title = tk.Label(root, text="Hybrid Cryptosystem Simulator",
                font=("Arial", 18, "bold"), fg="#38bdf8", bg="#0f172a")
title.pack(pady=10)

# Steps
steps = tk.Label(root,
    text="Step 1: Generate Keys → Step 2: Enter Message → Step 3: Encrypt → Step 4:
Decrypt",
    fg="white", bg="#0f172a", font=("Arial", 10))
steps.pack()

# Input
tk.Label(root, text="Enter Plaintext:", fg="white", bg="#0f172a").pack(anchor="w",
padx=20)
input_text = scrolledtext.ScrolledText(root, height=5, width=100)
input_text.pack(pady=5)

# Buttons
btn_frame = tk.Frame(root, bg="#0f172a")
btn_frame.pack(pady=10)

tk.Button(btn_frame, text="Generate Keys", command=generate_keys,
          bg="#2563eb", fg="white", width=15).grid(row=0, column=0, padx=5)
```

```
tk.Button(btn_frame, text="Encrypt", command=encrypt_message,
        bg="#22c55e", fg="black", width=12).grid(row=0, column=1, padx=5)

tk.Button(btn_frame, text="Decrypt", command=decrypt_message,
        bg="#ef4444", fg="white", width=12).grid(row=0, column=2, padx=5)

tk.Button(btn_frame, text="Load File", command=load_file,
        bg="#3b82f6", fg="white", width=12).grid(row=0, column=3, padx=5)

tk.Button(btn_frame, text="Clear", command=clear_all,
        bg="#f97316", fg="white", width=10).grid(row=0, column=4, padx=5)

# Status
status_label = tk.Label(root, text="Status: Waiting...", fg="yellow", bg="#0f172a")
status_label.pack(pady=5)

# Outputs
tk.Label(root, text="Encrypted AES Key (Base64):", fg="white",
bg="#0f172a").pack(anchor="w", padx=20)
key_box = scrolledtext.ScrolledText(root, height=3, width=100)
key_box.pack(pady=5)

tk.Label(root, text="Encrypted Ciphertext (Base64):", fg="white",
bg="#0f172a").pack(anchor="w", padx=20)
cipher_box = scrolledtext.ScrolledText(root, height=3, width=100)
cipher_box.pack(pady=5)

tk.Label(root, text="Decrypted Message:", fg="white", bg="#0f172a").pack(anchor="w",
padx=20)
output_box = scrolledtext.ScrolledText(root, height=3, width=100)
output_box.pack(pady=5)

# Operation Log
tk.Label(root, text="Operation Log:", fg="#38bdf8", bg="#0f172a").pack(anchor="w",
padx=20)
log_box = scrolledtext.ScrolledText(root, height=8, width=100)
log_box.pack(pady=5)

# Run
```

```
root.mainloop()
```

Backend Code:(backend.py)

```
import os
import base64
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes

def generate_keys():
    private_key = rsa.generate_private_key(
        public_exponent=65537,
        key_size=2048
    )
    return private_key, private_key.public_key()

def encrypt_message(public_key, message):
    aes_key = os.urandom(32)
    iv = os.urandom(16)

    cipher = Cipher(algorithms.AES(aes_key), modes.CFB(iv))
    encryptor = cipher.encryptor()
    encrypted_data = encryptor.update(message.encode()) + encryptor.finalize()

    encrypted_key = public_key.encrypt(
        aes_key,
        padding.OAEP(
            mgf=padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None
        )
    )

    return (
        base64.b64encode(encrypted_data).decode(),
        base64.b64encode(encrypted_key).decode(),
        encrypted_data,
        encrypted_key,
        iv
```

```
    )

def decrypt_message(private_key, encrypted_data, encrypted_key, iv):
    decrypted_key = private_key.decrypt(
        encrypted_key,
        padding.OAEP(
            mgf=padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None
        )
    )

    cipher = Cipher(algorithms.AES(decrypted_key), modes.CFB(iv))
    decryptor = cipher.decryptor()
    original = decryptor.update(encrypted_data) + decryptor.finalize()

    return original.decode()
```

sample.txt:

Hello this is my hybrid cryptography projects

README.txt:

PROJECT TITLE: Hybrid Cryptosystem Simulator

DESCRIPTION:
This project demonstrates a hybrid cryptographic system that combines symmetric (AES) and asymmetric (RSA) encryption to ensure secure data transmission.

OBJECTIVE:
To provide both speed and security by using AES for encrypting data and RSA for encrypting the AES key.

FEATURES:
- AES encryption for fast data encryption
- RSA encryption for secure key exchange
- Decryption process to retrieve original message

TECHNOLOGY USED:
- Python
- Cryptography Library

HOW TO RUN:

1. Open terminal in project folder
2. Run command: python hybrid_crypto.py
3. Enter message when prompted

OUTPUT:

Displays encrypted message, encrypted AES key, and decrypted message.

CONCLUSION:

Hybrid cryptography provides better security and efficiency compared to using only symmetric or asymmetric encryption alone.

Appendix B: Glossary of Terms

Term	Definition
AES	Advanced Encryption Standard — a symmetric block cipher approved by NIST in 2001.
RSA	Rivest-Shamir-Adleman — an asymmetric public-key cryptosystem based on integer factorization.
Symmetric Encryption	Encryption where the same key is used for both encryption and decryption.
Asymmetric Encryption	Encryption using a mathematically related key pair — public and private keys.
Hybrid Cryptography	A system combining symmetric and asymmetric encryption for optimal security and performance.
Session Key	A temporary symmetric key generated for a single communication session.
IV (Initialization Vector)	A random value used to ensure that identical plaintexts produce different ciphertexts.
OAEP	Optimal Asymmetric Encryption Padding — a probabilistic, secure padding scheme for RSA.
CFB Mode	Cipher Feedback Mode — an AES mode that turns a block cipher into a stream cipher.
Ciphertext	The encrypted, unreadable form of a plaintext message.
Plaintext	The original, readable form of a message before encryption.
Key Encapsulation	The process of encrypting a symmetric key using an asymmetric key.
Public Key	The key in an asymmetric pair that can be freely shared and used for encryption or verification.
Private Key	The secret key in an asymmetric pair, used for decryption or signing.
SHA-256	Secure Hash Algorithm 256-bit — a cryptographic hash function used in OAEP and digital signatures.
MGF1	Mask Generation Function 1 — a pseudo-random function used in OAEP padding.

TLS/SSL	Transport Layer Security / Secure Sockets Layer — protocols for secure internet communication.
PGP	Pretty Good Privacy — a data encryption and decryption program using hybrid cryptography.
GUI	Graphical User Interface — a visual interface allowing user interaction through graphical elements.
Tkinter	Python's standard GUI library for creating desktop applications.

Appendix C: Algorithm Flowcharts

C.1 Encryption Pipeline Flowchart

The following describes the logical flow of the encryption process:

START → User enters plaintext message → Generate RSA-2048 key pair → Generate AES-256 session key (os.urandom(32)) → Generate 128-bit IV (os.urandom(16)) → Encrypt plaintext with AES-256-CFB → Encrypt AES session key with RSA-OAEP public key → Display: Ciphertext + Encrypted AES Key

The encryption pipeline illustrates the sequence of operations performed to securely transform a user's plaintext message into an encrypted format suitable for secure transmission. This process combines both symmetric and asymmetric cryptographic techniques to achieve efficiency and security.

The process begins when the **user inputs a plaintext message**, which serves as the original data to be protected. Once the input is received, the system initiates cryptographic operations by generating an **RSA-2048 key pair**. This key pair consists of a public key, used for encryption, and a private key, used for decryption. In the current implementation, this key pair is generated dynamically at runtime.

Next, the system generates a **random AES-256 session key** using a secure random function such as os.urandom(32). This 256-bit key is used for symmetric encryption, which is computationally efficient and suitable for encrypting the actual message content. Alongside this, a **128-bit Initialization Vector (IV)** is also generated using os.urandom(16). The IV ensures that identical plaintext messages produce different ciphertexts, thereby enhancing security and preventing pattern analysis.

The plaintext message is then encrypted using the **AES-256 algorithm in CFB (Cipher Feedback) mode**. This mode allows encryption of data in a streaming manner and does not require padding, making it suitable for variable-length inputs. The result of this step is the **ciphertext**, which represents the encrypted version of the original message.

To securely transmit the AES session key, the system encrypts it using the **RSA-OAEP (Optimal Asymmetric Encryption Padding) public key**. This ensures that only the intended recipient, who possesses the corresponding private key, can decrypt and retrieve the AES key. This combination of AES and RSA forms a **hybrid encryption scheme**, leveraging the speed of symmetric encryption and the security of asymmetric encryption.

Finally, the system outputs the following components:

- The **ciphertext** (encrypted message)
- The **encrypted AES session key**

In a complete implementation, the IV would also be included with the output, as it is required for decryption (though it is not kept secret). The resulting encrypted data can then be safely transmitted over insecure channels.

This pipeline ensures **confidentiality of the message**, efficient processing, and a scalable approach to encryption suitable for modern secure communication systems.

C.2 Decryption Pipeline Flowchart

The decryption pipeline represents the reverse process of encryption, where the encrypted data is transformed back into its original plaintext form. This process ensures that only an authorized party with access to the correct private key can successfully recover the message.

The process begins when the system **receives the encrypted AES session key**, which was previously secured using the RSA public key during the encryption phase. To recover the original symmetric key, the system uses the **RSA-OAEP private key** to decrypt this encrypted session key. Since RSA-OAEP provides strong padding and security, it ensures that the decryption process is resistant to common cryptographic attacks.

Once the decryption is successful, the system **recovers the original AES-256 session key**, which is essential for decrypting the actual message. This step is critical because the AES key is the only means to transform the ciphertext back into readable plaintext. Next, the system **receives the ciphertext along with the Initialization Vector (IV)**. The IV, although not secret, must match the one used during encryption to ensure correct decryption. It helps maintain the integrity of the decryption process by preserving the original encryption state.

Using the recovered AES session key and the IV, the system proceeds to **decrypt the ciphertext using the AES-256 algorithm in CFB (Cipher Feedback) mode**. This mode allows the ciphertext to be processed in a stream-like manner, making it efficient and suitable for data of varying lengths. During this step, the encrypted data is transformed back into its original plaintext format.

After successful decryption, the system **recovers the original plaintext message** exactly as it was entered during the encryption phase. The final step involves **displaying the decrypted message to the user**, confirming that the process has been completed successfully.
The pipeline concludes with the **END** state, indicating that the secure decryption process is complete.

This decryption workflow ensures that:

- Only authorized users can access the original message (confidentiality)
- The hybrid encryption mechanism (RSA + AES) is correctly reversed
- The system maintains consistency and reliability in secure data recovery

Appendix D: Security Analysis

D.1 Confidentiality Analysis

The system provides strong confidentiality guarantees through two layers of encryption. AES-256 with a random IV ensures that the ciphertext reveals no information about the plaintext, even when the same message is encrypted multiple times. The random IV is critical — reusing an IV with the same key in CFB mode would allow an attacker to recover plaintext by XOR-ing two ciphertexts, but os.urandom() ensures IV uniqueness with overwhelming probability.

RSA-OAEP provides semantic security for the key encapsulation step. The probabilistic nature of OAEP means that the same AES key encrypted multiple times produces a different ciphertext each time, preventing chosen-ciphertext attacks and ensuring that an attacker cannot determine whether two encrypted key blobs correspond to the same session key.

D.2 Limitations and Known Weaknesses

The current implementation has the following security considerations that are acceptable for a simulator but would need addressing in a production system:

- The RSA key pair is regenerated on each application launch, meaning encrypted data cannot be decrypted in a later session without key persistence.
- The IV is not stored alongside the ciphertext in the current implementation, meaning that in a production system, the IV would need to be transmitted with the ciphertext (it is not secret, just unique).
- There is no authentication mechanism — the system provides confidentiality but not integrity or authenticity. Adding HMAC or AES-GCM would address this.
- The private key is held in the same process as the public key, which is appropriate for simulation but not for real-world asymmetric key usage.
- The system does not implement proper **secure key storage**. In a real-world application, private keys should be stored in secure hardware modules (e.g., HSMs) or protected using strong encryption and access controls.
- • There is no **protection against replay attacks**. An attacker could capture and resend previously transmitted ciphertext, which may be accepted as valid without verification mechanisms like timestamps or nonces.
- • The implementation lacks **input validation and error handling**, which could lead to unexpected behavior or vulnerabilities such as crashes or injection attacks in a production environment.
- • The random number generation used for key or IV creation may not be **cryptographically secure**. A production system must rely on secure random number generators to prevent predictability.

- • There is no **performance optimization** for handling large data. RSA is computationally expensive and not suitable for encrypting large payloads directly; hybrid encryption (RSA + symmetric encryption) should be enforced properly.
- • The system does not include **key rotation or expiration policies**, which are essential for maintaining long-term security and limiting the impact of compromised keys.
- • No **logging or monitoring mechanisms** are implemented. In a real system, security events should be logged and monitored for intrusion detection and auditing.
- • The application does not defend against **side-channel attacks** (e.g., timing attacks), which can potentially leak sensitive information during cryptographic operations.
- • There is no **user authentication or access control**, meaning any user can access cryptographic operations without restriction.
- • The system assumes a **trusted environment**, which is unrealistic in production where attackers may have partial system access.

D.3 Compliance with Cryptographic Best Practices

Best Practice	Implemented?	Details
Cryptographically secure random number generation	Yes	os.urandom() used for all key/IV generation
Secure padding for RSA	Yes	OAEP with SHA-256 (not PKCS#1 v1.5)
Sufficient key sizes	Yes	RSA-2048 and AES-256 meet current standards
Unique IV per encryption	Yes	New IV generated for each encrypt_text() call
No hard-coded keys or IVs	Yes	All keys generated dynamically at runtime
Production-grade library	Yes	cryptography library (OpenSSL binding)
Authenticated encryption	No	Future enhancement: AES-GCM or HMAC-SHA256